

Flutter Development Environment Setup (2026)

Dr M AOUE
Android Studio Edition

Context. This guide provides a standardized Flutter setup for teams using Android Studio, ensuring consistent development environments across Windows, macOS, and Linux with version control, validation, and troubleshooting included.

Quick Start (5-minute setup)

1. Install the latest stable Android Studio
2. Clone Flutter: `git clone https://github.com/flutter/flutter.git -b stable`
3. Add Flutter to PATH and run `flutter doctor`
4. Install Flutter/Dart plugins in Android Studio
5. Run `flutter create my_app && cd my_app && flutter run`

1 Prerequisites

One consistent setup for Windows, macOS, and Linux that plays perfectly with Android Studio, avoids version drift, and enables hot reload, testing, profiling, and CI from day one.

Requirement	Version / Note
OS	Windows 10/11, macOS 13.5+, Ubuntu 22.04+
RAM	16 GB minimum (32 GB recommended for emulator + IDE)
Storage	50+ GB SSD free (SDKs, emulators, caches)
Android Studio	Latest stable (Quail 4, Sept. 2026, or newer)
Git	2.34+ (needed for Flutter version management)
Internet	Required for SDK downloads and dependencies

Tip: Use `git clone` for Flutter (not Snap/Homebrew/Chocolatey) to keep team control over versions.

2 Install Git

```
1 git --version
```

```
2
```

Listing 1: Check Git

If missing:

Windows (PowerShell):

```
1 winget install --id Git.Git -e
```

```
2
```

macOS:

```

1  xcode-select --install # includes Git
2  # or:
3  brew install git
4

```

Ubuntu:

```

1  sudo apt update && sudo apt install git -y
2

```

Basic config:

```

1  git config --global user.name "Your Name"
2  git config --global user.email "you@example.com"
3  git config --global init.defaultBranch main
4

```

3 Install Android Studio (use Embedded JDK)

1. Download the **latest stable Android Studio**: <https://developer.android.com/studio>
2. Install with defaults.
3. Gradle JDK will point to the **Embedded JDK** bundled with Studio (JDK 17 or newer, depending on the release). No separate JDK is required.

Note: You don't need JAVA_HOME for Flutter/Studio. Only set it if other tools require it—and point it to Studio's embedded JDK (17 or newer). *Without Android Studio* (command-line SDK only) there is no embedded JDK: install JDK 21 (for example `winget install --id Microsoft.OpenJDK.21 -e --source winget`) and tell Flutter where it is with `flutter config --jdk-dir "<JDK folder>"`.

4 Install Flutter SDK (via git clone)

Choose a clean path without spaces.

macOS/Linux

On Ubuntu, install Flutter's prerequisite packages first:

```

1  sudo apt-get update -y && sudo apt-get install -y curl git unzip xz-utils
2  zip libglu1-mesa

```

On macOS, run `xcode-select --install` first (it includes Git). Prefer an Apple Silicon Mac: Flutter is deprecating Intel Macs.

```

1  mkdir -p ~/dev && cd ~/dev
2  git clone https://github.com/flutter/flutter.git -b stable
3  echo 'export PATH="$HOME/dev/flutter/bin:$PATH"' >> ~/.zshrc
4  source ~/.zshrc
5

```

Windows (PowerShell)

```

1  mkdir C:\dev; cd C:\dev
2  git clone https://github.com/flutter/flutter.git -b stable
3  [Environment]::SetEnvironmentVariable(
4  "Path",
5  "C:\dev\flutter\bin;" + [Environment]::GetEnvironmentVariable("Path","User")
6  ,
7  "User"
8  )

```

Verify:

```

1  flutter --version
2

```

Windows + WSL2: Keep your project on C:\ (Windows filesystem). Run the emulator *outside* WSL for performance.

Warning: Choose installation paths without spaces or special characters to avoid build issues.

5 Configure Platforms & Accept Licenses

Android, web, and desktop targets are enabled by default in current Flutter releases.

```

1  # Show the current settings (enable or disable a target only if needed)
2  flutter config
3  # e.g. flutter config --no-enable-web
4
5  # Accept Android licenses
6  flutter doctor --android-licenses
7
8
9  # Pre-download artifacts (saves time later)
10 # iOS artifacts only on macOS:
11 flutter precache --android --web
12 # On macOS you can also add: --ios

```

6 Configure Android Studio

Plugins: Settings/Preferences → **Plugins** → install **Flutter** and **Dart** → restart.

Gradle JDK: Settings → Build, Execution, Deployment → Build Tools → **Gradle JDK:** **Embedded JDK.**

Android SDK: Settings → Appearance & Behavior → System Settings → **Android SDK**

- **SDK Platforms:** Android 16 (API 36) or newer (the minimum Flutter requires)
- **SDK Tools:**
 - Android SDK Build-Tools, Platform-Tools and Emulator (latest)
 - **Android SDK Command-line Tools (latest)** (*important*)
 - CMake and NDK (Side by side), as listed in Flutter's Android setup guide

Emulator (Device Manager):

- A recent Pixel device profile, API 36, with the system image matching your CPU (x86_64 on Intel/AMD, arm64 on Apple Silicon)
- Enable hardware acceleration (Hypervisor.framework on macOS, Windows Hypervisor Platform on Windows, KVM on Linux)
- Enable Quick Boot snapshots for fast startups

7 Verify Setup

From *Android Studio* → *Terminal*:

```
1 flutter doctor -v
2 adb devices
3 flutter devices
4
```

You should see green checks for Flutter, Android toolchain, Android Studio, and at least one device (emulator or physical).

Common fixes

- cmdline-tools missing → SDK Manager → install **Command-line Tools (latest)**.
- Licenses unknown → run `flutter doctor --android-licenses` again (as admin if needed).
- No devices → start AVD, enable USB debugging on phone, install OEM USB driver (Windows).

8 Team Consistency (version pinning)

Create `.flutter-version` in your repo (example):

```
1 3.47.0
2
```

macOS/Linux bootstrap

```
1 #!/bin/bash
2 if [ -f ".flutter-version" ]; then
3 v=$(cat .flutter-version)
4 pushd ~/dev/flutter >/dev/null
5 git fetch --tags
6 git checkout "tags/$v"
7 popd >/dev/null
8 echo "Flutter locked to $v"
9 fi
10
```

Windows PowerShell bootstrap

```
1 if (Test-Path .flutter-version) {
2     $v = Get-Content .flutter-version
3     Push-Location "C:\dev\flutter"
4     git fetch --tags
5     git checkout "tags/$v" --quiet
6     Pop-Location
7     Write-Host "Flutter locked to $v"
8 }
9
```

Alt: teams often use FVM; this `.flutter-version` approach stays tool-agnostic and CI-friendly.

9 Create & Run a Test App

```
1 flutter create --platforms=android,ios,web my_app
2 cd my_app
3 flutter run
4
```

In Android Studio: *File* → *Open* → *select my_app* → *choose device* → *Run*. Hot reload works on save.

Final Validation Checklist

- ☐ `git --version` & `flutter --version` OK
- ☐ `flutter doctor -v` all green
- ☐ Studio has **Flutter/Dart** plugins
- ☐ **Gradle JDK = Embedded JDK**
- ☐ **SDK Command-line Tools (latest)** installed
- ☐ Emulator boots fast (Quick Boot) or physical device detected
- ☐ `flutter run` works; hot reload on save

Troubleshooting Quick Reference

Symptom	Cause	Fix
<code>cmdline-tools</code> missing	Incomplete Android SDK	SDK Manager → Install "Command-line Tools (latest)"
Gradle JDK errors	JDK version mismatch	Studio Settings → Gradle JDK = Embedded JDK
Emulator slow	Missing hardware acceleration	Enable HVF/WHPX/KVM; use x86_64 images
<code>flutter</code> not found	PATH configuration	Re-add Flutter bin to PATH; restart terminal/IDE
No devices	ADB/driver issues	Run <code>adb devices</code> ; install OEM USB drivers
iOS precache fails	Platform mismatch	Run <code>flutter precache --ios</code> only on macOS
No JDK found (Android toolchain)	Android Studio not installed, or no JDK on PATH	Install JDK 17 or 21, then <code>flutter config --jdk-dir "<JDK folder>"</code>
Visual Studio missing components	Windows desktop workload not installed	Only needed for Windows desktop apps: add the "Desktop development with C++" workload, or ignore it for Android/web
CocoaPods issues	Outdated repos	<code>pod repo update &&</code> <code>pod install</code> (macOS only)

Next best steps

- Add Riverpod + GoRouter and your `analysis_options.yaml`.
- Create Run/Debug configurations in Studio per flavor:
 - Dev → `--flavor dev -t lib/main_dev.dart`
 - Prod → `--flavor prod -t lib/main_prod.dart --release`

- Start a background codegen terminal:

```
1 dart run build_runner watch --delete-conflicting-outputs
2
```

- Capture perf proof early: Studio Profiler (FPS $\geq$ 60, baseline memory $<$ 200 MB).

GitHub & CI/CD Setup Guide

Part 1: Push Your Project to GitHub

Objective: Securely version control your Flutter project and enable team collaboration through GitHub.

Step 1: Prepare Your Project for Git

```
1 # flutter create already generates a .gitignore; append the extras
2 printf '%s\n' '.env' '.env.local' '.env.production' >> .gitignore
3 printf '%s\n' '.DS_Store' 'Thumbs.db' '*.log' >> .gitignore
4
```

Step 2: Initialize Git and Create Initial Commit

```
1 # Initialize repository
2 git init
3
4 # Configure git (if not done globally)
5 git config user.name "Your Name"
6 git config user.email "your.email@example.com"
7
8 # Add all files and create initial commit (use multiple -m for a multiline
9 message)
10 git add .
11 git commit -m "feat: initial Flutter project setup" \
12 -m "Flutter 3.47.0 with the latest Android Studio" \
13 -m "Riverpod + GoRouter architecture" \
14 -m "Dev/Prod flavor configuration" \
15 -m "Analysis options and code quality setup"
```

Step 3: Create GitHub Repository

1. Go to <https://github.com/new>
2. **Repository name:** my-flutter-app (use kebab-case)
3. **Description:** *Flutter application with Riverpod, GoRouter, and multi-environment setup*
4. **Visibility:** Private (commercial) or Public (open source)
5. **Initialize:** *Do not* add README, .gitignore, or license
6. Click **Create repository**

Step 4: Connect and Push to GitHub

```
1 # Add the remote repository
2 git remote add origin https://github.com/your-username/my-flutter-app.git
3
4 # Rename main branch if needed (modern Git uses 'main')
5 git branch -M main
6
```

```
7  # Push with upstream tracking
8  git push -u origin main
9
10 # Verify
11 git log --oneline -5
12 git status
13
```

Step 5: Create Development Branch Structure

```
1  # Create and push the development branch
2  git checkout -b develop
3  git push -u origin develop
4
5  # Create a feature branch for new work
6  git checkout -b feature/add-authentication
7
8  # Standard workflow:
9  # 1) Work on feature branches
10 # 2) Merge to 'develop' for integration/testing
11 # 3) Merge 'develop' to 'main' for releases
12
```

This document is maintained by Dr. Mohamad AOUBE
Last updated: 2026-09-20